

第九章 Mask R-CNNとその関連技術

- ・ 物体検出・・・物体検出は特徴を抽出し、対象物の領域を切り出し、クラスを認識
- ・ 物体検出の流れ
 - ①物体の同定 (identification) : 画像の中で物体がどこにあるのか?
 - ②物体認識/分類 (classification) : 何の物体であるか?
- ・ 物体検出はバウンディングボックスを使用
- ・ セマンティックセグメンテーションとは
 - 物体領域を画素単位で切り出し、各画素にクラスを割り当てる手法
 - 工業検査や医療画像解析など、精密な領域分割に応用される
 - 要な学習データセットに、VOC2012とMSCOCOがある
- ・ 主要手法は、完全畳み込みネットワーク (FNC; Fully Convolutional Network)
 - 全ての層が畳み込み層、全結合層を有しない
 - 画素ごとにラベル付した教師データを与えて学習する→出力ノードが多数
 - 未知画像も画素単位でカテゴリを予測する
 - 入力画像のサイズは可変で良い
- ・ R-CNN (Regional CNN)
 - ・ 物体検出+物体認識のアルゴリズムの原形は R-CNN (Regional CNN)
 - ・ 物体検出タスクと物体認識タスクを順次に行う
 - 関心領域 (ROI; Region of Interest) を取り出す。
 - 候補領域の画像の大きさを揃えCNNにより特徴量を求める
- ・ R-CNNの発展版
 - 高速R-CNN (Fast R-CNN)・・・R-CNNより計算量を大幅に減少
 - YOLO (You Only Look Once) や SSD (Single Shot Detector)
- ・ インスタンス・セグメンテーション
 - 画像のピクセルをどのクラスに属するかに分類するのに加えて、検出したそれぞれの物体を見分け、クラスIDに加えインスタンスID(それぞれの物体のID)を予測する
- ・ セマンティックセグメンテーションとインスタンスセグメンテーションの違い
 - セマンティックセグメンテーションはピクセルをクラス分類するが、同じクラスの複数の物体があっても区別できない
 - インスタンスセグメンテーションは、同じクラスであっても個々の物体を区別することができる
 - 有名なアプローチはYOLACT・Mask R-CNN

- ・ Mask R-CNNとはFaster R-CNNを拡張したアルゴリズム

Mask R-CNNはインスタンスセグメンテーションに対応するので、Faster R-CNNの物体検出機能にセグメンテーションの機能を付加したイメージ
バウンディングボックス内の画素単位でクラス分類を行うため、物体の形も推定可能

- ・ Mask R-CNNの特徴

画像中の物体らしき領域とその領域にあるクラスを検出
→物体検出の結果として得られた領域についてのみセグメンテーションを行うことで
効率アップ

- ・ Mask R-CNNはFaster R-CNNと構造に類似点が多い

Faster R-CNN・・・物体クラス・バウンディングボックス

Mask R-CNN・・・物体クラス・バウンディングボックス・物体領域マスクの推定

- ・ RoI Pooling

Fast/Faster R-CNNではRoI Pooling を使用
畳み込み処理後の特徴マップから、region proposal領域を「固定サイズの特徴マップ」
として抽出

- ・ RoI PoolingとRoI Align

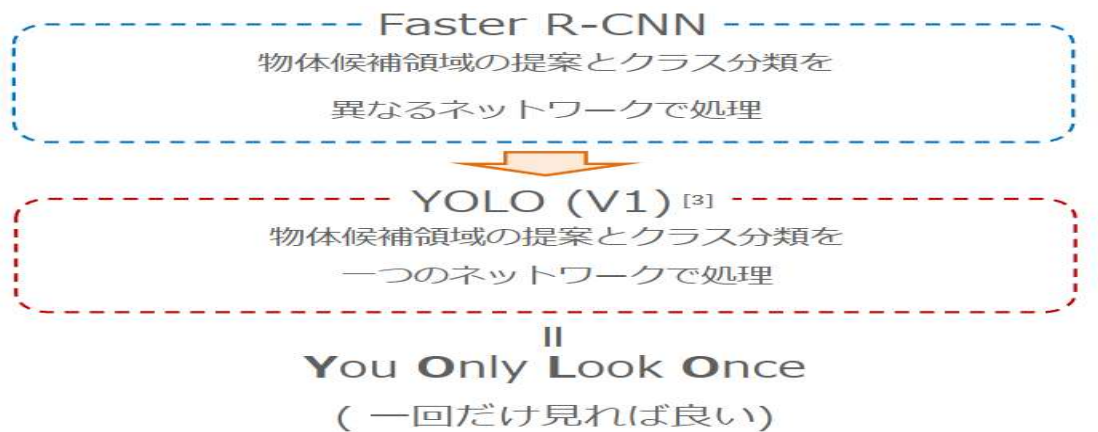
RoI Poolingの代わりに、Mask R-CNNでは新しい手法 RoI Align を導入
Alignment（位置合わせ）を重視したRoI (Region of Interest：関心領域)特徴
の作成がポイント→補完処理によって精度向上

- ・ RoI Alignの手順

1. $N \times N$ の特徴マップにしたい場合、 $N \times N$ の領域に分割する
2. その領域一つ一つについて4つの点を打つ
3. 一つ一つの点について、周りの四つのピクセルを使い、何らかの補間法で点の値を求める
4. 四つの点を一つにまとめる

第十章 Faster-RCNN, YOLO1

- ・ 物体候補領域の提案の処理にCNNを使用するRPN ((Region Proposal Network)を提案
→End-to-Endな処理が可能
- ・ 特徴マップ
入力された画像は、VGG16により特徴マップに変換される
- ・ Anchors・・・特徴マップに Anchor Points を仮視し、PointごとにAnchor Boxes
を作成する
- ・ RPNの出力
→各 Anchor Boxes を Grand Truth の Boxesと比較し、含まれているものが背景か
物体か、どれくらいズレてるか出力
- ・ PASCAL VOC データで検証→R-CNNやFast R-CNNよりmAPスコアが高い
- ・ YOLO (V1) | アイデア



- ・ 利点：
 - 高速な処理
 - 画像全体を一度に見るから、背景を物体と間違えることがない。
 - 汎化性が高い
- ・ 欠点
 - 精度はFaster-RCNNに劣る
- ・ YOLO (V1)の工夫
Grid cellの仕組みを利用・・・ $S \times S$ のグリッド（格子）に分割し、各グリッドセルが「自分の担当エリアに物体の中心があるか」を判断して予測する仕組み

- ・YOLO (V1)のネットワーク

各 Gridにおける各 バウンディングボックスの 中心, 高さ, 横(x, y, w, h), 信頼度スコアの 5 つと各クラスに対応する特徴マップを同時に出力

- ・従来手法のSelective Searchによる処理を改良し処理を高速化したい

→Faster R-CNN・・・CNNベースのRPNを提案し, End-To-Endなネットワークを可能

→YOLO・・・候補領域の提案とクラス分類を同時に行うネットワークの提案

<実践演習：YOLO>

```
from ultralytics import YOLO

... Creating new Ultralytics Settings v0.0.6 file
View Ultralytics Settings with 'yolo settings' or at '/root/.config/Ultralytics/settings.json'
Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dir=path/to/dir'. For help see https://docs.ultralytics.com/quickstart/#ultralytics-settings.

model = YOLO('yolov8n-cls.pt')

Downloading https://github.com/ultralytics/assets/releases/download/v8.3.0/yolov8n-cls.pt to 'yolov8n-cls.pt': 100% 5.3MB 309.8MB/s 0.0s

result = model.train(data="cifar10", epochs=3, imgsz=32)

... Ultralytics 8.3.243 Python-3.12.12 torch-2.9.0+cu126 CUDA:0 (Tesla
engine/trainer: agnostic_nms=False, amp=True, augment=False, auto_augm

WARNING Dataset not found, missing path /content/datasets/cifar10.
Downloading https://ultralytics.com/assets/cifar10.zip to '/content/d

model()

...
image 1/1 /content/sample_horse_image.png: 32x32 cat 0.27, horse 0.24, dog 0.13, airplane 0.08, truck 0.08, 3.0ms
Speed: 6.3ms preprocess, 3.0ms inference, 0.1ms postprocess per image at shape (1, 3, 32, 32)
[Ultralytics.engine.results.Results object with attributes:

boxes: None
keypoints: None
masks: None
names: {0: 'airplane', 1: 'automobile', 2: 'bird', 3: 'cat', 4: 'deer', 5: 'dog', 6: 'frog', 7: 'horse', 8: 'ship', 9: 'truck'}
obb: None
orig_img: array([[[[224, 227, 235],
```

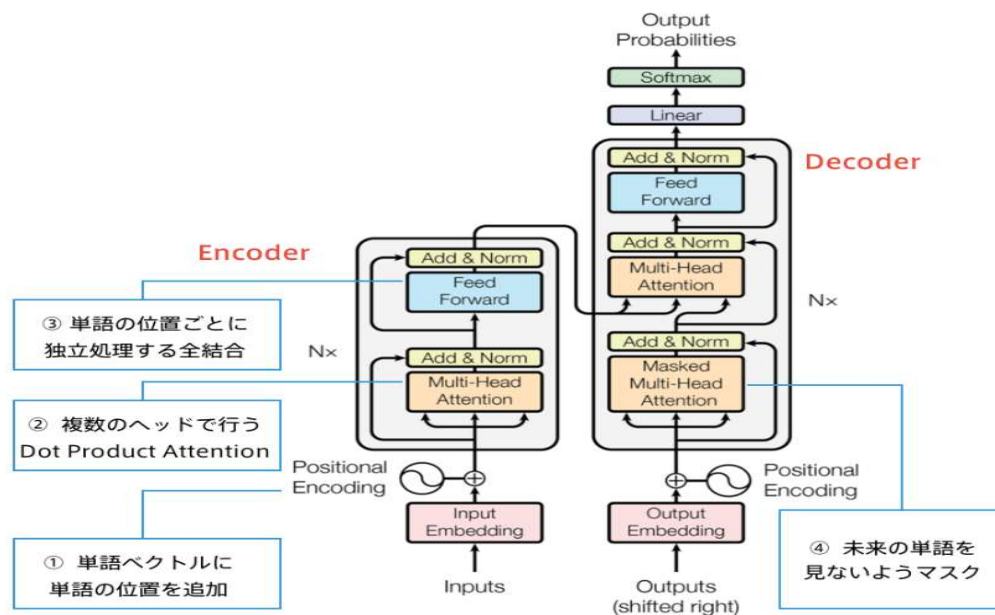
第十一章 FCOS

- ・多くの物体検出モデルは「アンカーボックス」に依存している。
- ・アンカーボックス・・・バウンディングボックスを出力する際にその候補となる出力ボックス
- ・アンカーボックスのデメリット：
 - ①ハイパーパラメータの設定に敏感である
 - サイズ・アスペクト比・数があるがパラメータによっては精度が4 % 上下する。
 - ②アンカーボックスのサイズやアスペクト比が固定されている
 - 向きや角度などによる形の変化が大きい物体に対応できない
 - 小型の物体に対応できない
 - ③ポジティブサンプルとネガティブサンプルのバランスが崩れる。
 - アンカーボックスのほとんどがネガティブサンプルのため、均衡が崩れ学習がうまくいなくなる。
- ・アンカーボックスを使わない。→アンカーボックス・proposalフリー
 - one-stageの手法である
- ・one-stage・・・バウンディングボックスの候補領域の選定とクラス判別を一括で行う
 - one-stageで済ませることで、計算コスト、学習時間を抑えられる
- ・COSでは上記の手法を使用
- ・アンカーフリー・・・YOLOv1・FCOS
- ・FCOSではFPN (FeaturePyramid Networks) を使用。
 - 複数のサイズの特徴マップを生成する
- ・特徴：低解像度の特徴は全体の特徴を捉えやすく、意味に強い
 - 高解像度の特徴は細かい部分に強いが、意味に弱い
- ・両方の良いところを両立させたのがFPN
- ・FPN・・・「大きさの違う物体を、同時にうまく見つけるための仕組み」
 1. CNNの途中途中の層（浅い～深い）を取り出す
 2. 深い層の特徴マップを上に拡大（アップサンプリング）
 3. それを浅い層の特徴マップと 足し合わせる
 - これを階段状（ピラミッド状）に繰り返します。

第十二章 Transformer

- ・ 2017年6月に登場
- ・ RNNを使わない
- ・ 必要なのはAttentionだけ
- ・ 当時のSOTAをはるかに少ない計算量で実現
- ・ 英仏 (3600万文) の学習を8GPUで3.5日で完了

- ・ 主要モジュール



- ・ Self-Attentionが肝
- ・ 入力を全て同じにして学習的に注意箇所を決めていく。
- ・ 全単語を同時に眺めて、「どこと関係が深い」を計算
- ・ Multi-Head Attention・・・複数の視点で同時に文章を分析
重みパラメタの異なる 8 個のヘッドを使用

- ・ Decoder
- ・ Encoderと同じく6層
- ・ 自己注意機構
→生成単語列の情報を収集
- ・ Encoder-Decoder attention
→入力文の情報を収集
- ・ Add (Residual Connection)
→入出力の差分を学習させる
- ・ Norm (Layer Normalization)
→各層においてバイアスを除く活性化関数への入力を平均 0、分散 1 に正規化

第十三章 BERT

- Bidirectional Transformerをユニットにフルモデルで構成したモデル
- 事前学習タスクとして、マスク単語予測タスク、隣接文判定タスクを与える
- BERTからTransfer Learningを行った結果、8つのタスクでSOTA達成
- Fine-tuningアプローチの事前学習に工夫を加えた

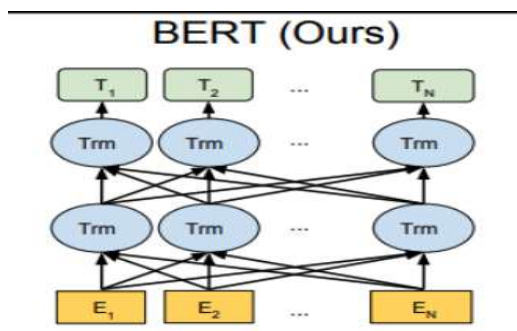
→双方向Transformer

tensorを入力としtensorを出力

モデルの中に未来情報のリークを防ぐためのマスクがそんざいしない。

従来のような言語モデル型の目的関数は採用できない。（カンニングになるため）

事前学習タスクにおいて工夫する必要がある



- 文の前後を同時に理解する双方向性を実現しており、BERTは従来の片方向性モデルに比べて文脈を深く理解できる
- 文の前後を同時に理解する双方向性を実現しており、BERTは従来の片方向性モデル
Masked Language Model（MLM）とNext Sentence Prediction（NSP）の2つの手法を使用。

第十四章 GPT

- ・ GPT-nモデル – 事前学習と転移学習
 - ・ 巨大な文章のデータセット（コーパス）を用いて事前学習（pre-trained）
 - 汎用的な特徴量を習得済みで、転移学習（transfer learning）に使用可能
 - ・ 転移学習を活用すれば、手元にある新しいタスク（翻訳や質問応答など）に特化したデータセットの規模が小さくても、高精度な予測モデルを実現できる
 - ・ 転用するにはネットワーク（主に下流）のファインチューニングを行う
 - ・ 代表的な事前学習モデルはBERTやGPT-nのモデルであり、事前学習と転移学習では全く同じモデルを使うことが特徴的
 - ・ 汎用的な学習済み自然言語モデルは、オープンソースとして利用可能なものもある
-
- ・ GPT-nモデル – GPTの特徴
 - ・ 特に、GPT-3のパラメータ数は1750億個にもなり、約45TBのコーパスで事前学習を行う
-
- ・ GPTの仕組み
 - ・ GPTの仕組みGPTの構造はトランスフォーマーを基本とし、「ある単語の次に来る単語」を予測し、自動的に文章を完成できるように、教師なし学習を行う
 - ・ 出力値は「その単語が次に来る確率」
 - ・ 学習前の1750億のパラメーターはランダムな値に設定され、学習を実行後に更新される
 - ・ 学習の途中で誤って予測をした場合、誤りと正解の間の誤差を計算しその誤差を学習する
-
- ・ GPT-3について報告されている問題点
 - ・ 社会の安全に関する課題
 - ・ 学習や運用のコストの制約
 - ・ 機能の限界（人間社会の慣習や常識を認識できないことに起因）
-
- ・ BERTとGPTの比較
 - ・ BERTとGPTの比較BERTは、Transformerのエンコーダー部分を使っている
 - ・ GPTはTransformerのデコーダー部分を使っている
 - ・ BERTは、新しいタスクに対してファインチューニングが必要
 - ・ GPT-3はファインチューニングをしない
 - ・ BERTは双方向Transformer
 - ・ GPTは単一方向のTransformer
 - ・ BERTは文の中のどの位置の単語もマスクされる可能性がありマスクした前の単語も後ろの単語にも注目する必要がある
 - ・ GPTは常に次の単語を予測するため、双方向ではない

- ・ Prompt based Learning の概要

→ Prompt Tuning : Prompt (指示) を学習によって最適化する手法

- ・ Prompt based Learning の利点

→ フェューショット学習 : 少数の例からでも学習が可能

→ 汎用性 : タスクごとに個別のモデルを作成して学習する必要がない

→ 柔軟性 : 特定のユースケースに合わせてモデルの微調整が可能

→ 効率性 : 迅速な学習により、学習と推論にかかる時間を短縮できる

→ パフォーマンスの向上 : より正確で信頼性の高い結果を期待できる

- ・ Prompt based Learning の展望

プロンプトデザインの改善 : より複雑な領域での活用

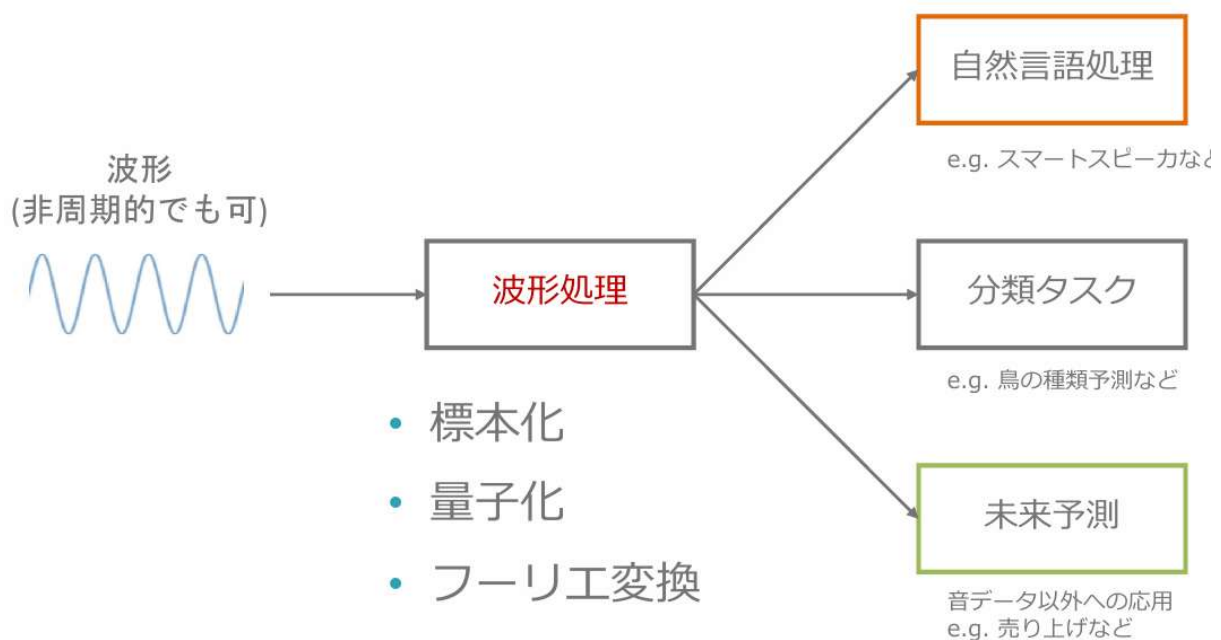
ベターフェューショット学習 : より少数の例からの学習

他のAI技術との統合 : より強力なモデルの作成

大規模言語モデルの進歩 : より強力で汎用的な学習

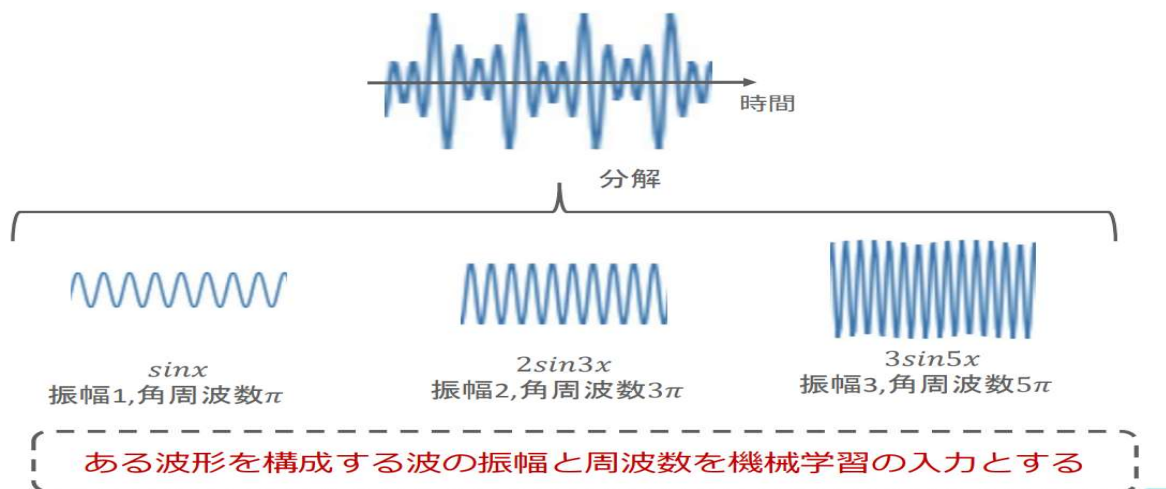
第十五章 機械学習のための音声データの扱い

- ・ 音が聞こえる仕組み
物体の振動による空気の振動
空気のない宇宙では音は聞こえない
- ・ 音波
空気の振動による音の波
- ・ 周波数と角周波数
周波数：一秒あたりの振動数(周期数)
角周波数：周波数を回転する角度で表現
- ・ 音波と機械学習

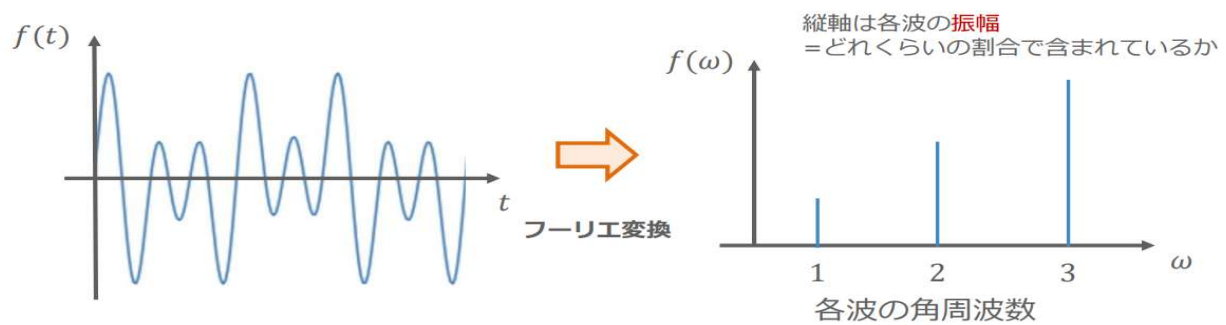


- ・ 波形の扱い
- ・ 波形の扱い標本化：連続時間信号を離散時間信号に変換
- ・ 量子化：等分した振幅にサンプルの振幅を合わせる
- ・ サンプリング周波数：1秒間で処理することができるサンプルの個数
- ・ サンプリングの法則
→ 周波数 h [kHz]の離散時間信号を測るには、最低 $2h$ [kHz]のサンプリング周波数が必要

- ・フーリエ変換とは
- ・目的：波形を機械学習の入力とするために行う。(標本化, 量子化と併用)
- ・前提：あらゆる波形(周期的・非周期的)は, 正弦波・余弦波を用いて表現できる
- ・振幅と周波数が分かれば, 波の特性がわかる！

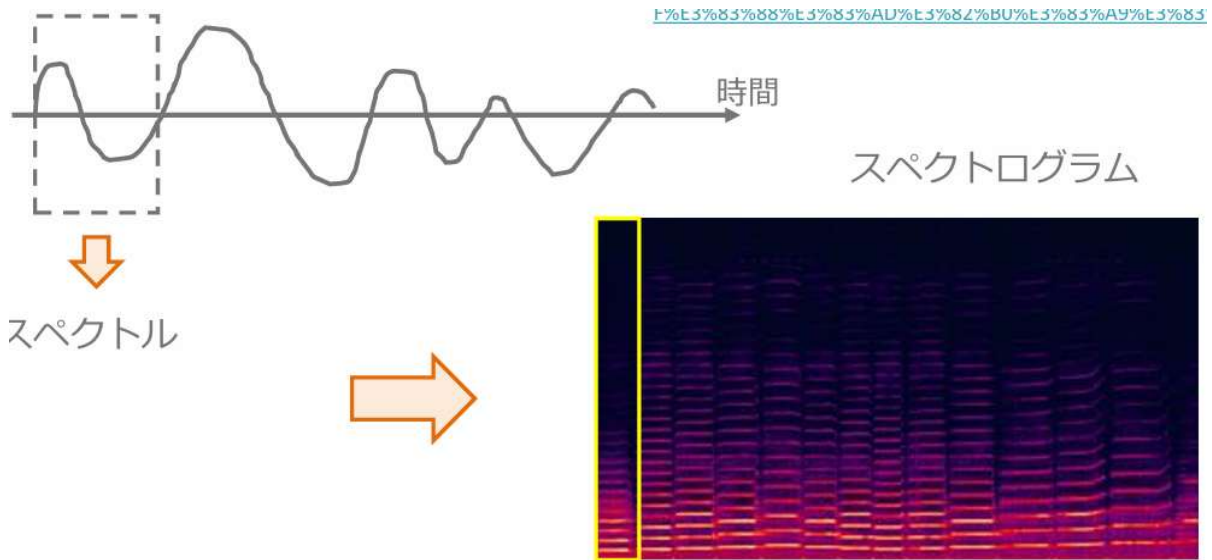


- ・定義：ある波形 $f(t)$ から振幅・角周波数を表す関数 $F(\omega)$ に変換する作業



この図を**スペクトル**と呼ぶ

- ・ スペクトログラム
- ・ 目的：現実的である非周期音声データの分析
- ・ 窓関数：波形を特定の時間区間(窓)で区切る



- 窓関数

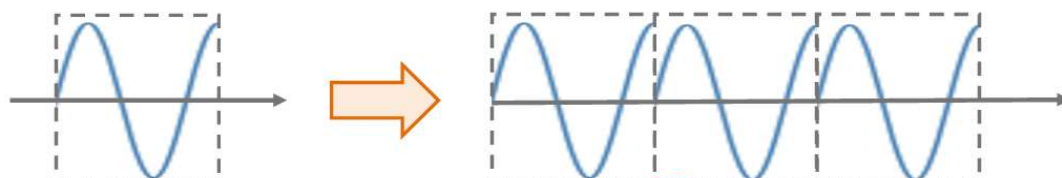
窓の大きさが問題となる

元の波形



波形が大きく異なる

ダメな例

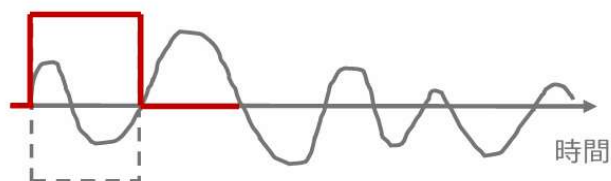


窓 1 つ

乖離している

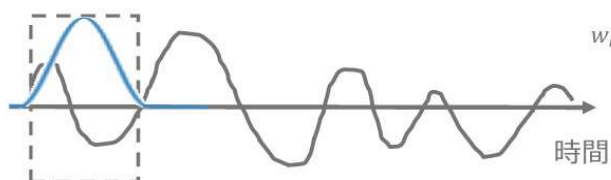
- 窓のつなぎ目を滑らかにするために区間ごとの波形関数にかける関数

- 普通の窓：矩形窓



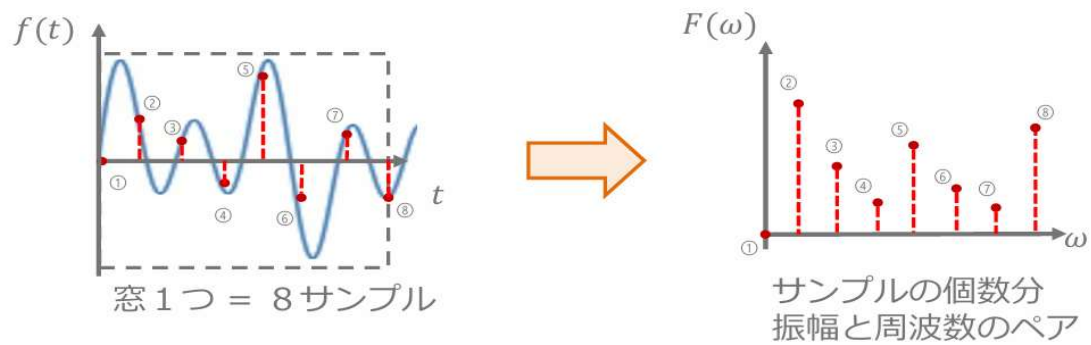
$$w_r[n] = \begin{cases} 1 & (n = 1, 2, \dots, N-1) \\ 0 & (\text{otherwise}) \end{cases}$$

- ハミング窓



$$w_h[n] = \begin{cases} 0.54 - 0.46 \cos \frac{2\pi n}{N} & (n = 1, 2, \dots, N-1) \\ 0 & (\text{otherwise}) \end{cases}$$

- ・ DFTとFFTにおけるサンプル数
- ・ DFT：離散フーリエ変換



- ・ FFT：高速フーリエ変換
- ・ 窓のサンプルのうち、偶数番目と奇数番目を別々に測定
- ・ 窓に含まれるサンプルN個(Nは2の冪乗：8,16,32, ..., 1024, 2048, ...)
→ N/2個のサンプルを測定→高速化
- ・ メル尺度：人間の聴覚に基づいた尺度
周波数の低い音に対して敏感で、周波数の高い音に対して鈍感であるという性質がある
- ・ 逆フーリエ変換：振幅・周波数から元の波形を構築する作業
- ・ ケプストラム：フーリエ変換したものの絶対値の対数を逆フーリエ変換して得られるもの

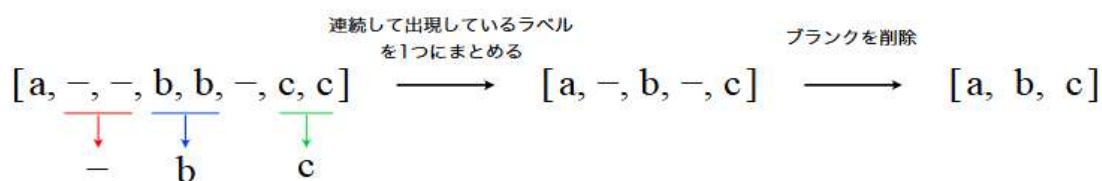
第十六章 CTC

- CTC (Connectionist Temporal Classification)
 - ディープニューラルネットワーク (DNN) だけで音響モデルを構築する手法
 - ブランク (blank) と呼ばれるラベルの導入
 - 前向き・後ろ向きアルゴリズム (forward-backward algorithm) を用いた DNN の学習

• ブランクの導入

フレーム単位のラベル系列を最終的なテキスト系列に変換

1. 連続して出現している同一ラベルを 1 つのラベルにまとめる
2. ブランク「-」を削除する



• ブランクを導入する理由

- 1 つは、同一ラベルが連続するテキスト系列を表現するため
- 厳密にアライメントを決定させないため

• 前向き・後ろ向きアルゴリズムを用いた RNN の学習

入力音声系列 x に対して縮約後の出力テキスト系列が $l = [a, b, c]$ となる事後確率

$$\begin{aligned}
 P(l|x) &= P([a, -, b, b, b, c, -, -]|x) + P([- , -, a, -, b, b, -, c]|x) + \dots \\
 &= \sum_{\pi \in \mathcal{B}^{-1}(l)} P(\pi|x)
 \end{aligned}$$

音声認識モデルは、この $P(l|x)$ が最大となるようなテキスト系列 l を音声認識結果として出力

• 前向き確率 $\alpha_t(s)$

始点からフレーム t 、拡張ラベル s の頂点に到達するまでの全パスの確率の総和

$$\alpha_t(s) \equiv \sum_{B(\pi_{1:t})=l_{1:[s/2]}^*} \prod_{t'=1}^t y_{\pi_{t'}}^{t'}$$

• 後ろ向き確率 $\beta_t(s)$

フレーム t 、拡張ラベル s の頂点から終点まで到達する全パスの確率の総和

$$\beta_t(s) \equiv \sum_{B(\pi_{t:T})=l_{[s/2+1]:s}^*} \prod_{t'=t}^T y_{\pi_{t'}}^{t'}$$

$$\beta_t(s) \equiv \sum_{\mathcal{B}(\pi_{t:T})=l_{[s/2]:|t*|}^*} \prod_{t'=t}^T y_{\pi_{t'}}$$

- ・ $P(l^*|x)$ [そして CTC 損失関数 $LCTC = -\log P(l^*|x)$] を計算するためには、前向き確率 $\alpha_t(s)$ と後ろ向き確率 $\beta_t(s)$ さえ用意すればよい

第十七章 GAN

- Generative Adversarial Network (敵対的生成ネットワーク) の略
→2つのAIが競い合いながら、どんどん本物そっくりのデータを作り出す仕組み

- Generator: 乱数からデータを生成
- Discriminator: 入力データが真データ(学習データ)であるかを識別
→2プレイヤーのミニマックスゲーム

- 2プレイヤーのミニマックスゲーム
→人が自分の勝利する確率を最大化する作戦を取る
→もう一人は相手が勝利する確率を最小化する作戦を取る
→GANでは価値関数Vに対し、Dが最大化、Gが最小化をお行う。
→GANの価値関数はバイナリクロスエントロピー

- 最適化方法

➤ Generatorのパラメータ θ_g を固定

- 真データと生成データを m 個ずつサンプル
- θ_d を勾配上昇法(Gradient Ascent)で更新

$$\frac{\partial}{\partial \theta_d} \frac{1}{m} [\log[D(x)] + \log[1 - D(G(z))]]$$

➤ Discriminatorのパラメータ θ_d を固定

- 生成データを m 個ずつサンプル
- θ_g を勾配降下法(Gradient Descent)で更新

$$\frac{\partial}{\partial \theta_g} \frac{1}{m} [\log[1 - D(G(z))]]$$

- なぜGeneratorは本物のようなデータを生成するのか？
生成データが本物とそっくりな状況とは
■ $p_g = p_{data}$ であるはず
→価値関数が $P_g = P_{data}$ の時に最適化されていることを示せばよい。
→二つのステップにより確認する
 - 1.Gを固定し、価値関数が最大値をとるときの $D(x)$ を算出
 - 2.上記の $D(x)$ を価値関数に代入し、Gが価値関数を最小化する条件を算出

< 実践演習 : dcgan.ipynb >

- ✓ 学習済みモデルを用いて画像生成

```
[12] model.generate_img()
✓ 1秒

... Generate Images
Saved All Generated Data at /content/drive/MyDrive/pythonライブラリ基礎/DNN_code_colab_day4_ver2/DNN_code_colab_day4_ver2/notebook/4_10_code_dcgan/exp/exp_2/samples/infer
```

- DCGAN(Deep Convolutional GAN)
 - GANを利用した画像生成モデル
 - いくつかの構造制約により生成品質を向上
- Generator
 - Pooling層の代わりに転置畳み込み層を使用
 - 最終層はtanh、その他はReLU関数で活性化
- Discriminator
 - Pooling層の代わりに畳み込み層を使用
 - Leaky ReLU関数で活性化
- 共通事項
 - 中間層に全結合層を使わない
 - バッチノーマライゼーションを適用
- DCGANのネットワーク構造
- Generator
 - 転置畳み込み層により乱数を画像にアップサンプリング
- Discriminator
 - 畳み込み層により画像から特徴量を抽出し、最終層をsigmoid関数で活性化
- 応用技術紹介
- Fast Bi-layer Neural Synthesis of One-Shot Realistic Head Avatars
 - 1枚の顔画像から動画像(Avatar)を高速に生成するモデル
- 生成モデル
- 課題：生成モデルであるGANの学習は不安定であり、モード崩壊などの問題を生じやすい
 - モード崩壊 (mode collapse)
 - 識別器をだましやすいく特定の画像形式（モード）を学習
 - 生成画像の多様性が消滅
 - 学習が不安定
 - 勾配消失問題
 - 意味のない損失
 - 識別器と生成器は学習ごとに更新される
 - 学習ごとの損失関数と画像品質に相関なし

- ・ Wasserstein GAN (WGAN)

分布間の距離を定義する Wasserstein Distance を GAN の損失関数に導入して
安定な学習を可能

Wasserstein GAN (WGAN) の損失関数は、学習に応じて生成画像の質の向上と
ともに安定して減少

- ・ CycleGAN . . . GAN を用いて画像様式を変換する生成モデル

- ・ ペア画像不要
- ・ 教師なし学習
- ・ 双方向変換

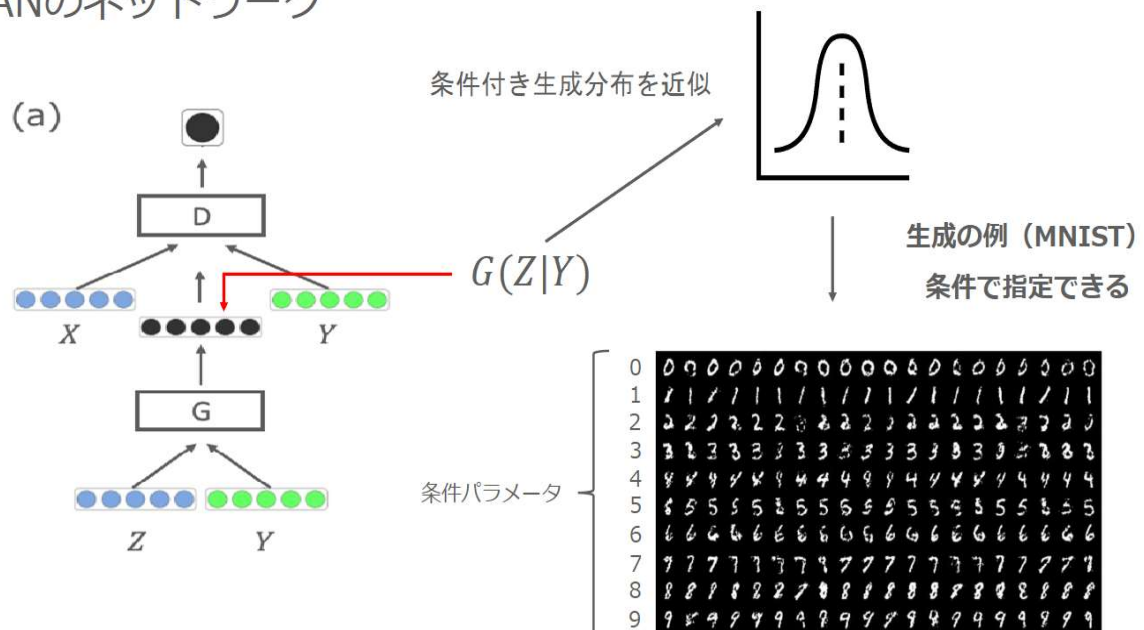
- ・ CycleGAN は、 2 つの画像集合間を双方向に変換でき、 4 つのニューラルネットワーク構造を持

- ・ データセット：異なる特徴を持つ画像集合
- ・ 2 つの識別機
- ・ 2 つの生成器
- 敵体的損失・復元性損失・同一性損失

第十八章 Conditional GAN

Conditional GAN：画像生成時に条件パラメータを与えることで、生成したい画像のクラスを指定できる一方で、従来のGANでは生成する画像のクラスは指定できない

CGANのネットワーク



第19章 pix2pix

- ・ pix2pixは「入力画像そのもの」を条件
- ・ 役割

CGANと同様の考え方

条件としてラベルではなく画像を用いる

条件画像が入力され、何らかの変換を施した画像を出力する

画像の変換方法を学習

- ・各プレイヤーの役割（条件画像x）

Generator : 条件画像 x をもとにある画像 $G(x, z)$ を生成

Discriminator : (条件画像 $x \rightarrow$ Generatorが生成した画像 $G(z|x)$)の変換と

(条件画像 $x \rightarrow$ 真の変換が施された画像 y) の変換が正しい変換かどうか識別する

- ・工夫 1 : U-Net . . . Generatorに使用、物体の位置を抽出
- ・工夫 2 : L1正則化項の追加 . . . Discriminatorの損失関数に追加
- ・工夫 3 : PatchGAN . . . 条件画像をパッチに分けて、各パッチにPix2pixを適応

第二十章 A3c

- ・ Asynchronous Advantage Actor-Critic (A3C)

強化学習の学習法の一つ; DeepMindのVolodymyr Mnih(ムニ)のチームが提案

特徴: 複数のエージェントが同一の環境で非同期に学習すること

- ・ “A3C”の名前の由来: 3つの“A”

Asynchronous → 複数のエージェントによる非同期な並列学習

Advantage → 複数ステップ先を考慮して更新する手法

Actor → 方策によって行動を選択

Critic → 状態価値関数に応じて方策を修正

- ・ Actor-Critic: 行動を決めるActor (行動器) を直接改善しながら、方策を評価する
Critic (評価器) を同時に学習させるアプローチ

- ・ 並列分散エージェントで学習を行うA3Cのメリット

① 学習が高速化

② 学習を安定化

→ A3Cはオンポリシー手法であり、サンプルを集めるエージェントを並列化することで自己相関を低減することに成功

- ・ A3Cの難しいところ

Python言語の特性上、非同期並列処理を行うのが面倒

パフォーマンスを最大化するためには、大規模なリソースを持つ環境が必要

第二十一章 距離学習 (Metric Learning)

- ・ 「似ている／似ていないを測る“物差しそのもの”を学習させる」技術

距離学習は少し哲学的で、「この2つはどれくらい近い存在か?」を数値化

- ・ モデルは入力 (画像・文章・音声など) をベクトル (数値の並び) に変換

- ・ 同じクラスに属する (= 類似) サンプルから得られる特徴ベクトルの距離は小さくする

- ・ 異なるクラスに属する (= 非類似) サンプルから得られる特徴ベクトルの距離は大きくする

- ・ 特徴ベクトルの属する空間は埋め込み空間

- ・ Siamese Network

同じネットワークを2つ使い、「この2つは同じ? 違う?」を学習。

- ・ Triplet Loss

3つ組で学習: アンカーサンプル (基準) ・ 類似サンプル ・ 非類似サンプル

第二十二章 MAML (メタ学習)

- ・ MAMLが解決したい課題

深層学習モデルの開発に必要なデータ量を削減したい

- ・ 深層学習モデル開発に必要なデータ量が必要、人手のアノテーションコスト

- ・ 少ないデータだと過学習が発生しやすい

→少ないデータで学習させたい

- ・ 得られた知識を未知の問題に適用する メタ学習 (meta-learning)

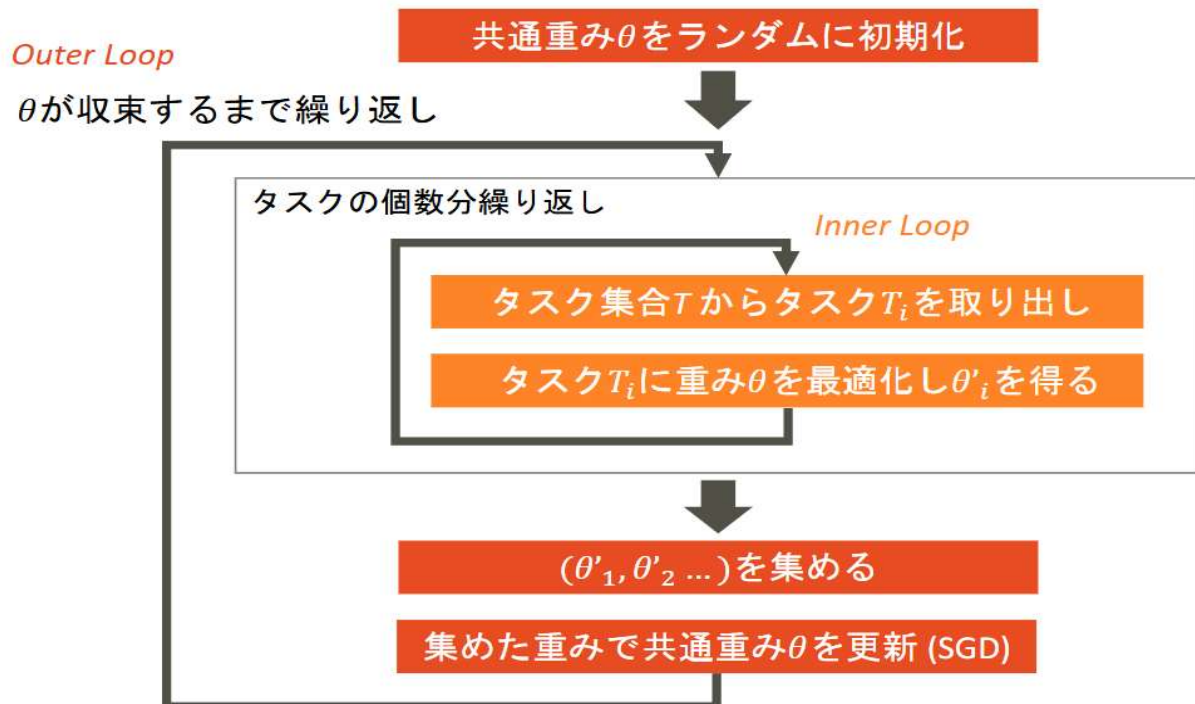
- ・ MAMLのコンセプト：タスクに共通する重みを学習し、新しいモデルの学習に活用

- ・ 特徴：

Model-Agnostic・・・微分可能である以外、モデルや損失関数の具体的な形式を仮定しない

Task-Agnostic・・・回帰、分類、強化学習など、様々なタスクに適用できる

- ・ MAMLの学習手順：タスクごとの学習を行った結果を共通重みに反映させ学習



- ・ MAMLの効果：Few-Shot learningで既存手法を上回る精度を実現

- ・ MAMLの課題：タスクごとの学習と共通パラメータの学習で計算量が多い

- ・ 計算コストを削減する改良案 (近似方法)

First-order MAML: 2次以上の勾配を無視し計算コストを大幅低減

Reptile: Inner loopの逆伝搬を行わず、学習前後のパラメータの差を利用

第二十三章 グラフ畳み込み(GCN)

- ・ 畳み込み→フィルターをかけること
- ・ 画像のCNNがピクセルの近所を見るように、GCNはノード（点）の近所のノードを見る。グラフ構造で定義される
- ・ 各ノードの特徴を、隣接ノードの特徴と混ぜて更新する
1層目：自分の情報、直接つながっている隣人の情報を平均化重みづけする。
2層目：「隣人の隣人」まで情報が届く⇒層を重ねるほど、遠くの関係性
- ・ Spatial GCN・・・「グラフ」を空間的に考えて畳み込む
- ・ Spectral GCN・・・グラフをスペクトルに分解して畳み込む

第二十四章 Grad-CAM,LIME,SHAP

- ・ CAMとは直感的には出力層の重みを畳み込み特徴マップに投影することで、画像領域の重要性を識別すること
- ・ CAMの評価
CAMの精度を確認するために、CAMの結果からバウンディングボックスを生成し物体検出の評価指標を使い精度を比較している(Localization)
- ・ Grad-CAM
CNNモデルに判断根拠を持たせ、モデルの予測根拠を可視化する手法
名称の由来は”Gradient” = 「勾配情報」
最後の畳み込み層の予測クラスの出力値に対する勾配を使用
勾配が大きいピクセルに重みを増やす
- ・ Grad-CAMにおけるヒートマップの求め方
クラスcのスコアを y_c とし、k番目の特徴マップの座標(i,j)における値を A_{kij} とする
 y_c の A_k における勾配を、特徴マップの全要素について Global Average Poolingを施す
- ・ CAMはモデルのアーキテクチャにGAPがないと可視化できなかったのに
対し、Grad-CAMはGAPがなくても可視化できる。また、出力層が画像分類でなくてもよく、様々なタスクで使える

- ・ LIMEとは

特定の入力データに対する予測について、その判断根拠を解釈・可視化するツール

- ・ LIMEの働き方の詳細

- ・ 単純で解釈しやすいモデルを用いて、複雑なモデルを近似することで解釈を行う
- ・ LIMEへの入力は 1 つの個別の予測結果（モデル全体の近似は複雑すぎる）
- ・ 対象サンプルの周辺のデータ空間からサンプリングして集めたデータセットを教師データとして、データ空間の対象範囲内でのみ有効な近似用モデルを作成
- ・ 近似用モデルから予測に寄与した特徴量を選び、解釈を行うことで、本来の難解なモデルの方を解釈したことと見なす。

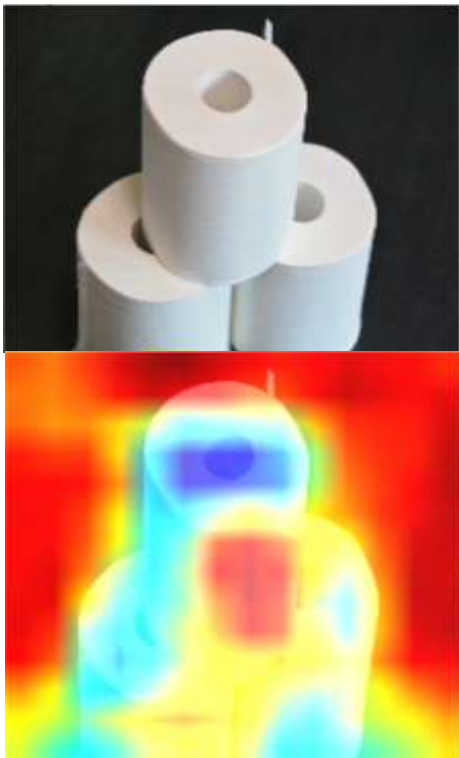
- ・ SHAPの解説

- ・ 協力ゲーム理論の概念であるshapley value（シャープレイ値）を機械学習に応用した
- ・ shapley valueが想定する状況：プレイヤーが協力し、それによって獲得した報酬を分配する。
→「限界貢献度」という概念を導入する

- ・ 以下の概念を機械学習に取り込む

協力ゲーム理論：協力して得た報酬を、貢献度が異なるプレイヤーにどう分配するか
機械学習：モデルから出力された予測値を、貢献度が異なる特徴量にどう分配するか

<実践演習：4_8_interpretability.ipynb>



第二十五章 Docker

- ・ コンテナ型仮想化

→仮想環境はハードウェア上で独立した複数の環境を実行する技術で、ホスト型、ハイパーバイザー型、コンテナ型の3種類が存在

- ・ Dockerはコンテナ型仮想化ソリューションで、事実上の標準として利用

Docker のアーキテクチャ： Docker エンジン・ Docker イメージ・ Docker コンテナ

- ・ Dockerは、アプリケーションの開発からデプロイメントまでの一貫性と効率性を向上させるために広く利用

→アプリケーションの開発とテスト・・・現代の開発の複雑さ

→本番環境でのデプロイ・・・コンテナの概念を導入することによる環境の隔離

→マイクロサービスアーキテクチャの実装・・・各マイクロサービスの独立したデプロイ

→環境の統一と再現性の確保・・・アプリケーションと依存関係のコンテナ化

開発、テスト、本番環境の動作の一貫性

- ・ Dockerを使用することで、GPUの計算リソースを効率的に活用したアプリケーションの開発とデプロイが簡単に行える。

→ Docker での GPU サポート

→GPU アプリケーションのコンテナ化

→NVIDIA Container Toolkit の特徴・・・GPU アクセラレーション、

エコシステムのサポート、自動設定

→深層学習モデルの開発とDocker・・・一貫した環境設定の簡易化、モデルの開発

やテストの一貫性

- ・ コンテナオーケストレーションは、大規模なアプリケーションなどで複数のコンテナ管理を実現するソリューション

- ・ 定義：コンテナのデプロイ、スケーリング、運用の自動化プロセス

- ・ 必要性：大規模アプリケーション・マイクロサービスの効果的な管理

コンテナダウン時の自動復旧

- ・ 代表的ツール：Kubernetes（オーケストレーションニーズ対応）